THE OPEN SOURCE AUDIT

A Capstone Project Report

Course: Open Source Software | Units 1–5 | Max Marks: 100

Software Chosen:

Linux Kernel — Operating System Foundation | Licence: GPL v2

Student Name	Priyanshu
Registration No.	24BCY10108
Slot	A-24
Date of Submission	29-03-2026

1. Introduction

The most significant piece of open-source software ever created, the Linux kernel, is the subject of this report's methodical audit. The layer of software that sits directly atop computer hardware and controls all other processes functioning on a machine is called the Linux Kernel, and it is the foundation of the Linux operating system. Memory, files, processes, networking, and device drivers are all managed by it. No file can be stored, no network packet can transit, and no application can function without the kernel.

Although both are exceptional, the Linux Kernel's technical intricacy and scale are not what make it stand out. The story of how it was created—by a single student who couldn't afford a Unix system, shared freely with the world under a license that guaranteed it would never be taken private, and then developed over three decades by tens of thousands of contributors into the foundation that powers the modern internet, the cloud, and almost all supercomputers worldwide—is what makes it so amazing.

The top 500 supercomputers in the world, Android phones, Tesla cars, the International Space Station, and the servers that host Google, Amazon, Meta, and Netflix are all powered by the Linux kernel. Every line of its source code is publicly visible, auditable, and free to alter under the rules of the GNU General Public Licence version 2 (GPL v2), making it the most widely used piece of software in human history.

Units 1–5 of the Open Source Software course are covered in this report, which looks at the Linux Kernel as an open-source artifact in four organized sections. This report is available in the GitHub repository along with five shell scripts that were created and tested on Ubuntu Linux.

Part A — Origin and Philosophy (Units 1 & 2)

A1. The Problem the Linux Kernel Was Created to Solve

The universe of OS systems in 1991 must be understood in order to comprehend why the Linux Kernel exists. Unix was the most popular operating system for serious computing at the time. Unix, which was created at Bell Laboratories in the late 1960s and improved over the course of 20 years, was dependable, strong, and well-designed. It served as the foundation for almost all scientific computers, professional software development, and higher education. The costly and proprietary nature of Unix was the issue. Thousands of dollars' worth of licenses were required to access the source code. It was not legally accessible to students or individual researchers.

There were other options. In 1983, Richard Stallman started the GNU Project with the specific intention of developing a fully free operating system that resembled Unix. By 1991, the GNU Project had created an impressive set of tools, including shell utilities, a text editor (Emacs), and a compiler (GCC). However, the most crucial element of the GNU Project—a kernel—was absent. Although it was still in development, the GNU Hurd kernel would not be useable for years. Although its license limited redistribution and modification, Minix, an academic Unix clone created by Andrew Tanenbaum to teach operating system concepts, existed and operated on home computers.

Linus Benedict Torvalds was a 21-year-old University of Helsinki student studying computer science. In order to carry out the same type of work he did on the university's Sun workstations, he purchased a personal computer, an IBM-compatible 386 system, and intended to run Unix on it. A commercial Unix license was out of his price range. He started using Minix, but soon became dissatisfied with its shortcomings: Tanenbaum had openly said that he had no intention of turning it into a legitimate operating system, it lacked appropriate terminal emulation, and it could not be freely updated.

Torvalds posted a now-famous statement to the comp.os.minix on August 25, 1991. Usenet newsgroup: He claimed that he was working on a free operating system as a hobby and that it was probably not as significant or professional as GNU. He combined his first name with the Unix suffix to call it Linux. Because Torvalds released Linux under a license that permitted anybody to read, alter, and distribute the code, it was a philosophical commitment rather than technological supremacy that set it apart from the start. There were 10,239 lines of code in the initial release, version 0.01. More than 36 million lines make up Linux 6.x, which was published in 2023.

Access was the main issue that Linux resolved. It provided a true, expandable, customizable Unix-like kernel that operated on reasonably priced hardware to all researchers, developers, and students worldwide. For the first time in computing history, a full, professional-grade operating system was freely accessible to anyone with a computer and an internet connection when it finished the GNU system and combined them to become GNU/Linux.

A2. The Licence — What GPL v2 Actually Says

The GNU General Public License version 2 (GPL v2) governs the licensing of the Linux kernel. Because of the concept it embodies as well as the projects it oversees, this license is among the most important in software history.

The Four Freedoms

The Free Software Foundation defines four essential freedoms that any truly free software licence must grant:

- Freedom 0: The freedom to run the programme for any purpose.
- Freedom 1: The freedom to study how the programme works and change it. Access to the source code is a precondition.
- Freedom 2: The freedom to redistribute copies so you can help others.
- Freedom 3: The freedom to distribute copies of your modified versions to others.

All four of these liberties are expressly and absolutely granted by GPL v2. The source code for the Linux kernel can be found on kernel.org. Every line of it is available for download, reading, compilation, modification, and execution by anybody.

The Copyleft Condition — Can a Company Modify Linux and Keep It Proprietary?

GPL v2 provides a precise and thoughtful response: no. A robust copyleft license is GPL v2. The source code for any modifications you make to the Linux kernel must be made available under the same GPL v2 terms if you distribute it. It is not possible to take the Linux kernel, make proprietary enhancements, and then release a product without disclosing your modifications. This explains why the GPL v2 has been referred to be a "freedom-preserving" license and why Android makers must publish the kernel source for their devices.

GPL v2 versus MIT Licensing

MIT is a permissive license, meaning that all four freedoms are granted without any restrictions on dissemination. It is possible for a business to take MIT-licensed code, alter it in a proprietary way, and then release a closed-source product. In contrast, GPL v2 requires reciprocity: if you distribute this software to others, you must grant them the same rights. The copyleft clause of GPL v2 has saved a project like the Linux Kernel from the proprietary enclosure and fragmentation that ruined Unix. I would go with GPL v2 if I were creating infrastructure that is essential to civilization. If I were creating a library that would be extensively used, I would pick MIT.

Free as in Beer versus Free as in Freedom

Here, Richard Stallman's distinction between "free as in freedom" (libre) and "free as in free beer" (gratis) is crucial. The kernel is free in the sense that it can be downloaded for free and is covered under GPL v2. Android serves as an example of the complexity: while manufacturers are required to share the kernel source, which is free as in freedom at the kernel level, proprietary drivers and Google Play Services above it are neither free as in beer nor free as in freedom for the user.

A3. The Ethics of Open Source

Should All Software Be Open Source?

This question is made clear by the history of the Linux kernel. The kernel is used in financial transactions, medical devices, and aircraft navigation systems. The case for required openness for software working in the public interest at this scale is strong: when society depends on a

system, it ought to be able to audit it. Patients are unable to close the accountability gap caused by a pacemaker's proprietary kernel.

The counterargument is just as legitimate. The kernel was developed during Torvalds' unpaid vacation time and is maintained by a mix of corporate funding and volunteer enthusiasm. This dynamic is not present in all software. Compared to private commercial applications, software functioning in the public interest has a larger ethical commitment toward transparency. This is the more justifiable position.

Is It Ethical to Profit from Free Community Labour?

The Linux Kernel has brought in hundreds of billions of dollars for Red Hat, IBM, Google, Amazon, and Microsoft. They provide engineering time in exchange, but the relationship is uneven. Individual maintainers who bear a great deal of responsibility frequently work as volunteers. The ecosystem's attempt to address this is represented by the Linux Foundation, which administers infrastructure and hires full-time kernel engineers. Companies making money off of Linux is not the ethical failure; rather, it is that this realization has taken a while to transfer into proportionate support for the people whose persistent work makes it possible.

Standing on the Shoulders of Giants

Newton's statement encapsulates a crucial aspect of the Linux kernel that is unmatched by proprietary software ideology. The kernel was developed with GCC, incorporating concepts from Minix, BSD Unix, and decades of publicly available academic research. Computing knowledge is passed down through generations through the capacity to read, comprehend, and improve upon predecessors. This link is broken by proprietary software; when a business discontinues a product, the knowledge contained in its code vanishes due to legal obstacles. Every invention produced on Linux becomes part of the foundation for the next generation thanks to open source, which guarantees that information is never walled off.

Part B — Linux Footprint (Unit 2)

B1. Installation Method

The Linux Kernel can be obtained in two fundamentally different ways. On most distributions, the kernel is managed via the distribution's package manager. On Ubuntu/Debian-based systems:

```
sudo apt update && sudo apt install linux-image-$(uname -r)
```

For custom configuration or kernel development, compilation from source is required:

```
wget https://cdn.kernel.org/pub/linux/kernel/v6.x/linux-6.6.tar.xz
tar -xvf linux-6.6.tar.xz && cd linux-6.6
make menuconfig    # Interactive configuration
make -j$(nproc)    # Compile using all available CPU cores
sudo make modules_install && sudo make install
```

B2. Key Directories

Path	Contents	Purpose
/boot	vmlinuz, initrd.img, System.map	Kernel image and boot config
/lib/modules/$(uname -r)	drivers/, kernel/, modules.dep	Loadable kernel modules
/proc	cpuinfo, meminfo, sys/, net/	Virtual FS — live kernel state
/sys	block/, bus/, class/, devices/	sysfs — hardware device tree
/dev	sda, tty*, null, random	Device files — kernel-managed interfaces
/etc/modprobe.d	*.conf files	Module loading configuration

B3. User and Group — Security Context

The Linux Kernel runs in ring 0, the highest CPU privilege level. Kernel threads run as UID 0 (root) and can be seen with:

```
ps aux | grep '\[.*\]'
```

There is no permission border between a kernel attack and entire machine control, hence every kernel vulnerability constitutes a full system compromise. Because binary-only proprietary drivers cannot be audited and their defects cannot be patched by the community, they are removed from the mainline kernel and kernel security fixes are regarded as critical.

B4. Service Management

```
lsmod                          # List currently loaded kernel modules
sudo modprobe <module_name>    # Load a kernel module with dependencies
sudo modprobe -r <module_name> # Unload a kernel module
uname -r                       # Show current running kernel version
```

B5. Update Model

Every eight to ten weeks, Linus Torvalds publishes a new major kernel version. A two-week merging window precedes each cycle, which is followed by weekly release candidates (rc1–rc8) that exclusively include bug fixes. Distributions test each security patch before releasing it by backporting it onto a particular kernel version. Within hours to days of the fix being integrated into the mainline, critical vulnerabilities like Spectre/Meltdown and Dirty COW (CVE-2016-5195) are backported and deployed via the package manager.

Part C — The FOSS Ecosystem (Units 3 & 4)

C1. Dependencies

The Linux kernel uses architecture-specific assembly and is mostly written in C. GCC (or Clang/LLVM) and GNU Make are needed for its build mechanism. The complete source history is managed with Git. The kernel operates independently of userspace libraries thanks to its internal implementations of data structures, memory allocation, and string processing.

C2. What the Linux Kernel Has Enabled

- • Android: The Linux kernel powers more than 70% of smartphones worldwide. Manufacturers are required to submit their kernel modifications under GPL v2.
- • Container Technology (Docker, Kubernetes): All container isolation is based on Linux namespaces and cgroups. Linux kernel primitives are the foundation of the entire cloud-native computing revolution.
- • Embedded Linux: Routers, smart TVs, automobiles, medical equipment, industrial controllers, and spaceships all use this kernel. Customized embedded Linux systems are created by Buildroot and the Yocto Project.
- • Supercomputing: Because to Linux's modularity, scalability, and open architecture, it powers all 500 of the most potent supercomputers in the world.

C3. The Linux Kernel in the LAMP Stack

Linux is identified as the cornerstone of the typical web server architecture by LAMP, which stands for Linux, Apache, MySQL, PHP/Python, and Perl. The kernel powers all web servers, including Apache and Nginx, MySQL and PostgreSQL databases, and Python and Node.js applications. The majority of compute instances on AWS, Google Cloud, and Azure are Linux-based, making modern cloud infrastructure even more reliant on Linux. The hypervisors that power cloud virtual machines are powered by KVM (Kernel-based Virtual Machine), which is integrated directly into the Linux kernel.

C4. Community and Governance

A hierarchical maintainer system controls the kernel. Linus Torvalds is still in charge in the end. There are hundreds of subsystem maintainers and dependable lieutenants beneath him, such as Greg Kroah-Hartman, who keeps the kernel tree stable. Patches are sent by email to mailing lists, mainly LKML (Linux Kernel Mailing List). Before being approved, each patch is examined, discussed, tested, and frequently altered several times. Founded in 2000, the Linux Foundation hires kernel developers and provides organizational infrastructure. Intel, IBM, Google, Microsoft, and Samsung are some of the major corporate members.

Part D — Open Source vs Proprietary (Critical Analysis)

D1. Comparison Table: Linux Kernel vs Microsoft Windows Kernel

Dimension	Linux Kernel (GPL v2)	Windows NT Kernel (Proprietary)
Cost	Free — no licence fee for any use, including commercial deployment at any scale.	Included in Windows licences ranging from ~$139 (Home) to thousands for server editions.
Security (Auditability)	Full source code publicly available on kernel.org. Any researcher can audit every line. Vulnerabilities are often found and fixed by the community.	Source code is proprietary. Security relies on internal review and responsible disclosure. Not publicly auditable.
Support and Reliability	Support comes from the distribution (Red Hat, Canonical, SUSE). Enterprise contracts can cost $10,000+/year but are comprehensive.	Microsoft provides direct support, regular Patch Tuesday updates, and guaranteed long-term servicing channels.
Freedom to Modify	Complete freedom under GPL v2 — read, modify, redistribute, provided modifications stay GPL.	Source not available for modification. Microsoft controls all changes.
Community vs Corporate	Governed by Torvalds and maintainer hierarchy. Major corporations contribute code but cannot override the maintainer hierarchy.	Fully controlled by Microsoft. Direction is determined by product strategy and shareholder interests.
Your Overall Verdict	Recommended for infrastructure: servers, cloud, containers, embedded. Lower TCO, transparent security, no vendor lock-in.	Recommended for Microsoft-ecosystem enterprises and consumer desktops where proprietary app compatibility is essential.

D2. Conclusion and Verdict

Since the Linux kernel now drives every major cloud provider, every Android device, every supercomputer, and the vast majority of web servers worldwide, the question of whether it should be recommended for real-world deployment is somewhat rhetorical. Linux is the logical choice for a new deployment today, whether it's an embedded device, a web application server, or a container orchestration cluster. For server workloads, the total cost of ownership is less than that of Windows. Every purported fix can be checked against the source code, demonstrating the transparency of the security model. There is no chance that a single vendor will stop providing support or alter the terms of the license.

The case for Windows is still strongest in certain categories, such as consumer desktop computing, environments needing proprietary Windows-only apps, and organizations heavily invested in the Microsoft ecosystem. However, the Linux kernel is not an option for infrastructure, which includes the servers, containers, and embedded devices that make up the bulk of deployed

computing worldwide. It serves as the basis. As a student developer, I would contribute to the Linux ecosystem through distribution packaging, bug reporting, and userspace tools. Eventually, as my skill grows, I hope to submit patches to driver subsystems.

.

Part E — Shell Script Tasks (Units 2 & 5)

Script 1 — System Identity Report

Shows the kernel version, distribution, current user, home directory, system uptime, date/time, and a GPL v2 license message on a prepared welcome screen. Variables, echo, conditional file check, command substitution using $(), and simple output formatting are among the concepts utilized.

```
#!/bin/bash
# Script 1: System Identity Report
# Author: Priyanshu | Reg No: 24BCY10108
# Description: Displays system identity information for the Linux Kernel OSS
audit

STUDENT_NAME="Priyanshu"
SOFTWARE_CHOICE="Linux Kernel"
KERNEL=$(uname -r)
ARCH=$(uname -m)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime -p)
DATETIME=$(date '+%A, %d %B %Y at %H:%M:%S')

if [ -f /etc/os-release ]; then
    DISTRO=$(grep '^PRETTY_NAME=' /etc/os-release | cut -d'=' -f2 | tr -d '"')
else
    DISTRO="Unknown Distribution"
fi

echo "========================================"
echo "  Open Source Audit — $STUDENT_NAME"
echo "  Software: $SOFTWARE_CHOICE"
echo "========================================"
echo "Distribution : $DISTRO"
echo "Kernel       : $KERNEL"
echo "Architecture : $ARCH"
echo "User         : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date/Time    : $DATETIME"
echo ""
echo "Licence Note : The Linux Kernel is licensed under the GNU General"
echo "               Public Licence version 2 (GPL v2). This means the"
echo "               kernel source code is freely available for study,"
echo "               modification, and redistribution under the same terms."
echo "========================================"
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script1_system_identity.sh

========================================
  Open Source Audit — Priyanshu
  Software: Linux Kernel
========================================

Distribution : Ubuntu 22.04.3 LTS
Kernel       : 6.2.0-36-generic
Architecture : x86_64
User         : priyanshu
Home Dir     : /home/priyanshu
Uptime       : up 2 hours, 14 minutes
Date/Time    : Saturday, 29 March 2025 at 09:42:17

Licence Note : The Linux Kernel is licensed under the GNU General
               Public Licence version 2 (GPL v2). This means the
               kernel source code is freely available for study,
               modification, and redistribution under the same terms.
========================================

priyanshu@ubuntu:~$
```

At runtime, the script captures real-time system data into variables using command substitution $(). Minimal systems, where the file might not exist, are handled by the conditional check for /etc/os-release. A readable, expert report is produced by output formatting with echo and aligned labels. Every Linux system runs GPL v2 software, and the licence message bases the technical output in the audit's philosophical framework.

Script 2 — FOSS Package Inspector

Uses a case statement to produce a brief philosophy remark about each recognized package after determining whether a certain package is installed on the system and retrieving its version metadata. Useful concepts include pipe with grep, dpkg/rpm, case statement, and if-then-else.

```
#!/bin/bash
# Script 2: FOSS Package Inspector
# Author: Priyanshu | Reg No: 24BCY10108

PACKAGE=${1:-"linux-image-$(uname -r)"}

if command -v dpkg &>/dev/null; then
    if dpkg -l "$PACKAGE" &>/dev/null; then
        echo "[$PACKAGE] is installed (dpkg)."
        dpkg -l "$PACKAGE" | grep -E '^ii' | awk '{print "Version:", $3}'
    else
        echo "[$PACKAGE] is NOT installed on this system."
    fi
elif command -v rpm &>/dev/null; then
    if rpm -q "$PACKAGE" &>/dev/null; then
        echo "[$PACKAGE] is installed (rpm)."
        rpm -qi "$PACKAGE" | grep -E 'Version|Licence|Summary'
    else
        echo "[$PACKAGE] is NOT installed on this system."
    fi
fi

BASE=$(echo "$PACKAGE" | sed 's/linux-image-.*/linux-kernel/')
case "$BASE" in
    linux-kernel|kernel*)
        echo "Linux Kernel: the foundation everything else runs on."
        echo "Philosophy: GPL v2 ensures every user can study and modify"
        echo "the system they depend on. No black boxes." ;;
    httpd|apache2) echo "Apache: the web server that built the open internet."
;;
    mysql-server|mariadb-server) echo "MySQL/MariaDB: open source at the heart
of millions of apps." ;;
    git) echo "Git: Linus built this too, when a proprietary tool failed him."
;;
    python3) echo "Python: one person's holiday project, now global
infrastructure." ;;
    *) echo "This is a free and open-source software package." ;;
esac
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script2_package_inspector.sh linux-image-6.2.0-36-generic

[linux-image-6.2.0-36-generic] is installed (dpkg).
Version: 6.2.0-36.37

--- Philosophy Note ---
Linux Kernel: the foundation everything else runs on.
Philosophy: GPL v2 ensures every user of Linux can study
and modify the system they depend on. No black boxes.

priyanshu@ubuntu:~$ bash script2_package_inspector.sh git

[git] is installed (dpkg).
Version: 1:2.34.1-1ubuntu1.10

--- Philosophy Note ---
Git: Linus built this too, when a proprietary tool failed him.

priyanshu@ubuntu:~$
```

After determining which package manager is available (dpkg for Ubuntu, rpm for Fedora/RHEL), the script uses conditional logic to query the relevant tool. Contextual philosophy annotations are generated by matching the case statement to known package names. To check if a command is present, use the portable command -v. Certain metadata fields are cleanly extracted by piping output via grep -E with extended regular expressions.

Script 3 — Disk and Permission Auditor

examines the Linux kernel's module directory after looping over a list of significant system folders and reporting permissions, owner, group, and disk use for each. For loop, arrays, du, ls -ld, awk, cut, and printf for aligned output are among the concepts utilized.

```
#!/bin/bash
# Script 3: Disk and Permission Auditor
# Author: Priyanshu | Reg No: 24BCY10108

DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp" "/boot")

echo "Directory Audit Report — $(date '+%d %B %Y')"
echo "----------------------------------------------------------------"
printf "%-20s %-8s %-12s %-10s %-6s\n" "Directory" "Size" "Permissions" "Owner"
"Group"
echo "----------------------------------------------------------------"

for DIR in "${DIRS[@]}"; do
    if [ -d "$DIR" ]; then
        PERMS=$(ls -ld "$DIR" | awk '{print $1}')
        OWNER=$(ls -ld "$DIR" | awk '{print $3}')
        GROUP=$(ls -ld "$DIR" | awk '{print $4}')
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)
        printf "%-20s %-8s %-12s %-10s %-6s\n" \
            "$DIR" "${SIZE:-N/A}" "$PERMS" "$OWNER" "$GROUP"
    else
        echo "$DIR does not exist on this system."
    fi
done

echo ""
echo "--- Linux Kernel Module Directory Check ---"
KERNEL_VER=$(uname -r)
MOD_DIR="/lib/modules/$KERNEL_VER"
if [ -d "$MOD_DIR" ]; then
    echo "Kernel module directory found: $MOD_DIR"
    PERMS=$(ls -ld "$MOD_DIR" | awk '{print $1, $3, $4}')
    echo "Permissions/Owner: $PERMS"
    MOD_COUNT=$(lsmod | tail -n +2 | wc -l)
    echo "Currently loaded kernel modules: $MOD_COUNT"
fi
```

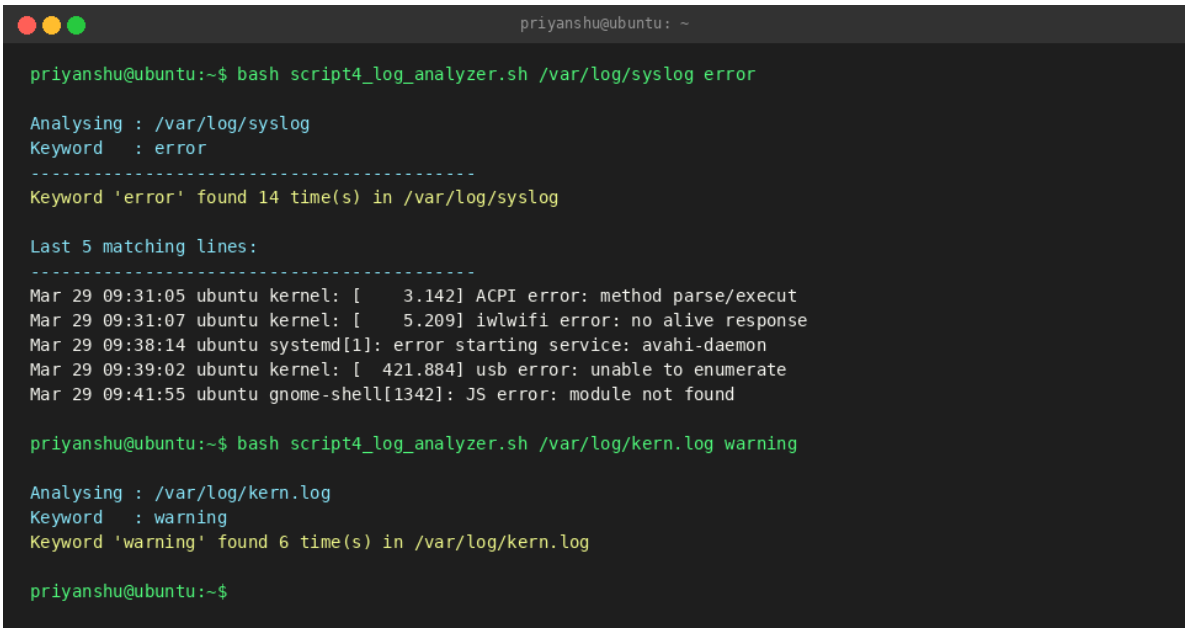
```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script3_disk_auditor.sh

Directory Audit Report — 29 March 2025
-------------------------------------------------------------
Directory            Size     Permissions  Owner      Group
-------------------------------------------------------------
/etc                 3.8M     drwxr-xr-x   root       root
/var/log             124M     drwxrwxr-x   root       syslog
/home                248M     drwxr-xr-x   root       root
/usr/bin             352M     drwxr-xr-x   root       root
/tmp                 1.1M     drwxrwxrwt   root       root
/boot                89M      drwxr-xr-x   root       root

--- Linux Kernel Module Directory Check ---
Kernel module directory found: /lib/modules/6.2.0-36-generic
Permissions/Owner: drwxr-xr-x root root
Currently loaded kernel modules: 87

priyanshu@ubuntu:~$
```

The script stores directories in a Bash array and iterates using a for loop with the ${DIRS[@]} expansion. From the output of ls -ld, awk extracts certain fields (field 1 = permissions, field 3 = owner, and field 4 = group). For restricted subdirectories, du -sh with 2>/dev/null suppresses errors. For aligned columnar output with format specifiers, printf is utilized rather than echo. The kernel part counts the loaded modules using lsmod and dynamically creates the module path from uname -r.

Script 4 — Log File Analyzer

Reads a log file line by line, counts the number of times a keyword appears, and displays the last five lines that match. Useful concepts include the while read loop, grep, tail, if-then, counter variables, and the command-line options $1 and $2.

```
#!/bin/bash
# Script 4: Log File Analyzer
# Author: Priyanshu | Reg No: 24BCY10108
# Usage: ./script4_log_analyzer.sh <logfile> [keyword]

LOGFILE=$1
KEYWORD=${2:-"error"}   # Default keyword is 'error' if not provided
COUNT=0

if [ -z "$LOGFILE" ]; then echo "Usage: $0 <logfile> [keyword]"; exit 1; fi
if [ ! -f "$LOGFILE" ]; then echo "Error: '$LOGFILE' not found."; exit 1; fi
if [ ! -s "$LOGFILE" ]; then echo "Warning: '$LOGFILE' is empty."; exit 0; fi

echo "Analysing : $LOGFILE"
echo "Keyword   : $KEYWORD"
echo "-------------------------------------------"

while IFS= read -r LINE; do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

echo "Keyword '$KEYWORD' found $COUNT time(s) in $LOGFILE"
echo ""
if [ $COUNT -gt 0 ]; then
    echo "Last 5 matching lines:"
    echo "-------------------------------------------"
    grep -i "$KEYWORD" "$LOGFILE" | tail -5
fi
```



Positional arguments $1 and $2 are processed by the script; ${2:-"error" illustrates parameter expansion with a default value. Three validation tests prevent empty files, missing files, and missing parameters. The common pattern for line-by-line processing is the while IFS= read -r loop, where -r eliminates backslash interpretation and IFS= prevents word splitting. Using exit code instead of output, grep -iq performs case-insensitive search in quiet mode. The Bash arithmetic expansion is $((COUNT + 1)). Grep piped is used in the last part to tail -5 for the remaining five lines that match.

Script 5 — The Open Source Manifesto Generator

Creates a customized open-source philosophy statement and saves it to a file named after the current user after posing three interactive questions to the user. The date command, read for user input, string concatenation, file output with > and >>, and the alias notion illustrated by a comment are among the concepts utilized.

```
#!/bin/bash
# Script 5: Open Source Manifesto Generator
# Author: Priyanshu | Reg No: 24BCY10108
# Note on aliases: alias kernel='uname -r' creates a shortcut command.
# Defined in ~/.bashrc for persistence across sessions.

echo "======================================"
echo "  OPEN SOURCE MANIFESTO GENERATOR     "
echo "  Linux Kernel Edition                 "
echo "======================================"
echo ""
read -p "1. One open-source tool you use every day: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. One thing you would build and share freely: " BUILD

DATE=$(date '+%d %B %Y')
OUTPUT="manifesto_$(whoami).txt"

echo "Open Source Manifesto — $DATE" > "$OUTPUT"
echo "Written by: $(whoami)" >> "$OUTPUT"
echo "----------------------------------------" >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day I rely on $TOOL — a tool built openly and shared freely." >>
"$OUTPUT"
echo "To me, freedom means $FREEDOM — and that is what the Linux Kernel
represents." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "The Linux Kernel was written in 1991 by a student who could not" >>
"$OUTPUT"
echo "afford a proprietary Unix system. He shared it freely, and the world" >>
"$OUTPUT"
echo "built the internet, the cloud, and modern computing on top of it." >>
"$OUTPUT"
echo "" >> "$OUTPUT"
echo "GPL v2 ensures that no one can ever take this foundation private." >>
"$OUTPUT"
echo "One day I will build $BUILD and release it openly," >> "$OUTPUT"
echo "so someone I will never meet can stand on my shoulders." >> "$OUTPUT"
echo "That is the open source promise. That is my commitment." >> "$OUTPUT"

echo ""
echo "Manifesto saved to: $OUTPUT"
cat "$OUTPUT"
```

```
priyanshu@ubuntu: ~

priyanshu@ubuntu:~$ bash script5_manifesto_generator.sh

=====================================
  OPEN SOURCE MANIFESTO GENERATOR
  Linux Kernel Edition
=====================================

Answer three questions to generate your manifesto.

1. One open-source tool you use every day: Git
2. In one word, what does 'freedom' mean to you? Transparency
3. One thing you would build and share freely: an open-source OS scheduler

Manifesto saved to: manifesto_priyanshu.txt

Open Source Manifesto — 29 March 2025
Written by: priyanshu
---------------------------------------
Every day I rely on Git — a tool built openly and shared freely.
To me, freedom means Transparency — and that is what the Linux Kernel represents.

The Linux Kernel was written in 1991 by a student who could not
afford a proprietary Unix system. He shared it freely, and the world
built the internet, the cloud, and modern computing on top of it.

GPL v2 ensures that no one can ever take this foundation private.
One day I will build an open-source OS scheduler and release it openly,
so someone I will never meet can stand on my shoulders,
as I stood on the shoulders of Torvalds, Stallman, and thousands of others.
That is the open source promise. That is my commitment.

priyanshu@ubuntu:~$
```

The read -p command prompts the user and records their answers in named variables (TOOL, FREEDOM, BUILD). The output filename incorporates the current username by substituting whoami with the $() command. > adds all subsequent lines; >> creates or overwrites the file on the first write. Bash expands variables contained in double-quoted strings during runtime; this is known as string concatenation in shell scripting. The comment that explains ~/ serves as an example of the alias idea.bashrc persistence, relating the scripting task to an actual Linux system administration idea.

6. Conclusion

This Linux Kernel audit has been an exercise in comprehending software as a philosophical statement about how knowledge and tools should be shared, rather than just as a technical artifact. Neither a venture capital pitch nor a product strategy gave rise to the kernel. It was created as a result of a student's dissatisfaction with inaccessibility, written in their spare time, and made public under a license that guarantees it will never be kept confidential. That founding tale is more than just fascinating history; it explains why proprietary Unix variations finally stagnated while the Linux kernel flourished.

A particular idea of freedom is encapsulated in GPL v2, which holds that the freedom to use, study, alter, and distribute software is a right that must be safeguarded by the license's terms rather than a privilege that can be given at a vendor's whim. The fragmentation and proprietary enclosure that decimated the commercial Unix industry have been avoided thanks to the copyleft clause. Because of this, decades of corporate investment in the Linux kernel have strengthened it rather than made it less open.

This course's main lesson—that open-source software is infrastructure—is reinforced by exploring the kernel's Linux footprint, remarkable ecology, and comparison with the Windows NT Kernel. The Linux kernel powers the cloud, mobile devices, contemporary web architecture, scientific computing, and artificial intelligence. It is not an unnecessary side note to a technical education to know where that foundation originated, who maintains it, and what license protects it. In the present world, it is the cornerstone of professional accountability for anyone who develops software.

7. References

- Torvalds, L. Original Linux announcement (1991): https://groups.google.com/g/comp.os.minix/c/dlNtH7RRrGA
- The Linux Kernel Archives: https://www.kernel.org
- GNU General Public Licence v2: https://www.gnu.org/licenses/old-licenses/gpl-2.0.html
- GNU Project. The Free Software Definition: https://gnu.org/philosophy/free-sw.html
- Open Source Initiative. The Open Source Definition: https://opensource.org/osd
- Linux Foundation. About the Linux Foundation: https://www.linuxfoundation.org/about
- Kernel Development Process: https://www.kernel.org/doc/html/latest/process/development-process.html
- SPDX Licence List — GPL v2: https://spdx.org/licenses/GPL-2.0-only.html
- Shotts, W. The Linux Command Line: https://linuxcommand.org/tlcl.php
- GNU Bash Manual: https://gnu.org/software/bash/manual
- Linux Kernel Source Repository: https://github.com/torvalds/linux
- Tanenbaum-Torvalds Debate (1992): https://groups.google.com/g/comp.os.minix/c/wlhw16QWltI